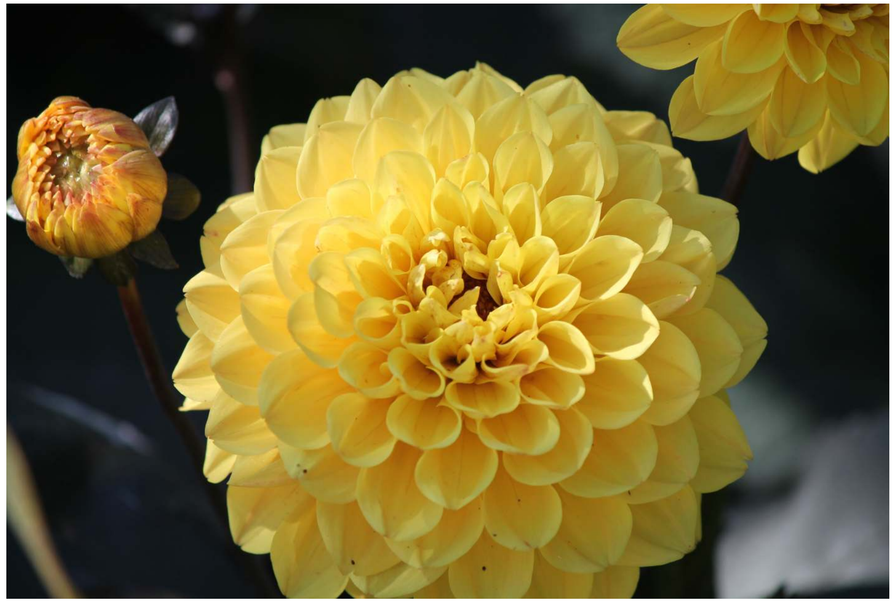
MACHINE LEARNING FOR SIGNAL PROCESSING



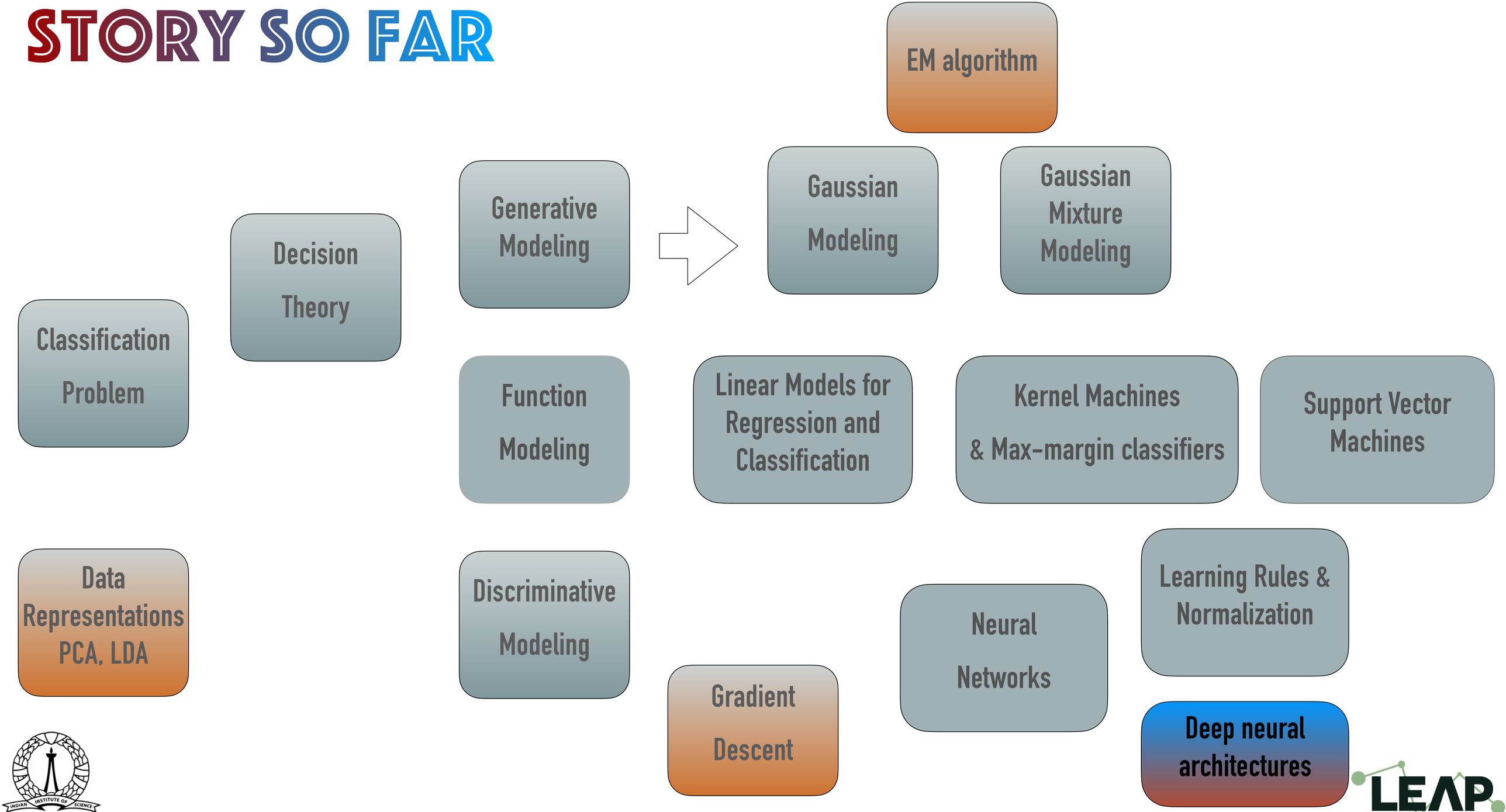
10-3-2025

Sriram Ganapathy
LEAP lab, Electrical Engineering, Indian Institute of Science,
sriramg@iisc.ac.in

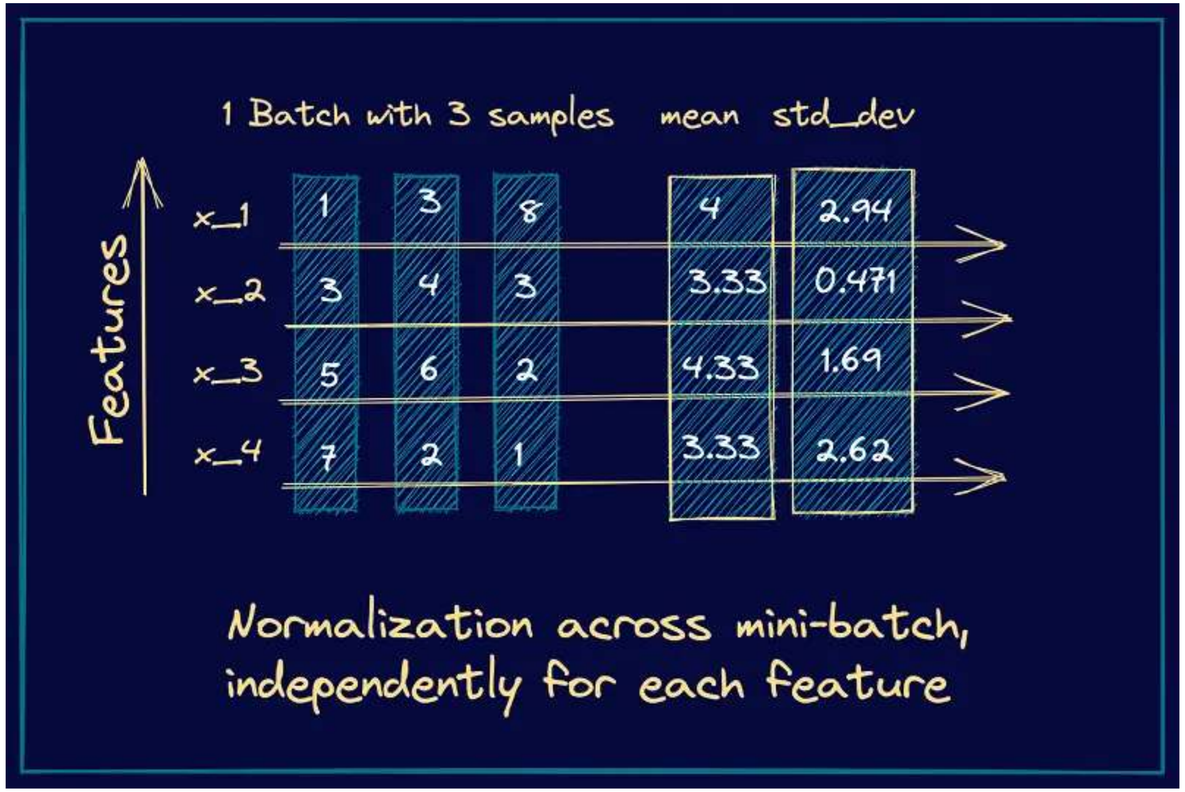
Viveka Salinamakki, Varada R.
LEAP lab, Electrical Engineering, Indian Institute of Science

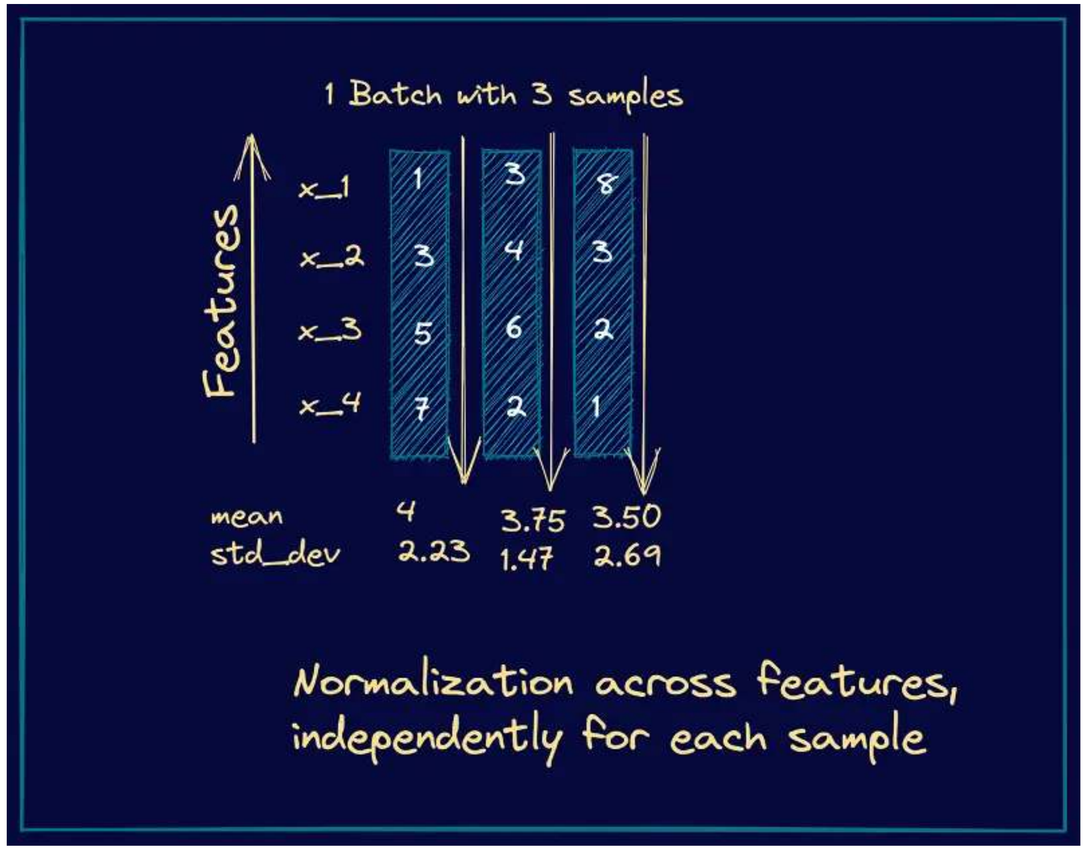
http://leap.ee.iisc.ac.in/sriram/teaching/MLSP25/

STORY SO FAR


EM algorithm
Generative Modeling
Gaussian Modeling
Gaussian Mixture Modeling
Decision Theory
Classification Problem
Function Modeling
Linear Models for Regression and Classification
Kernel Machines & Max-margin classifiers
Support Vector Machines
Data Representations PCA, LDA
Discriminative Modeling
Neural Networks
Learning Rules & Normalization
Gradient Descent
Deep neural architectures

COMPARING DIFFERENT NORMALIZATION





https://www.pinecone.io/learn/batch-layer-normalization/

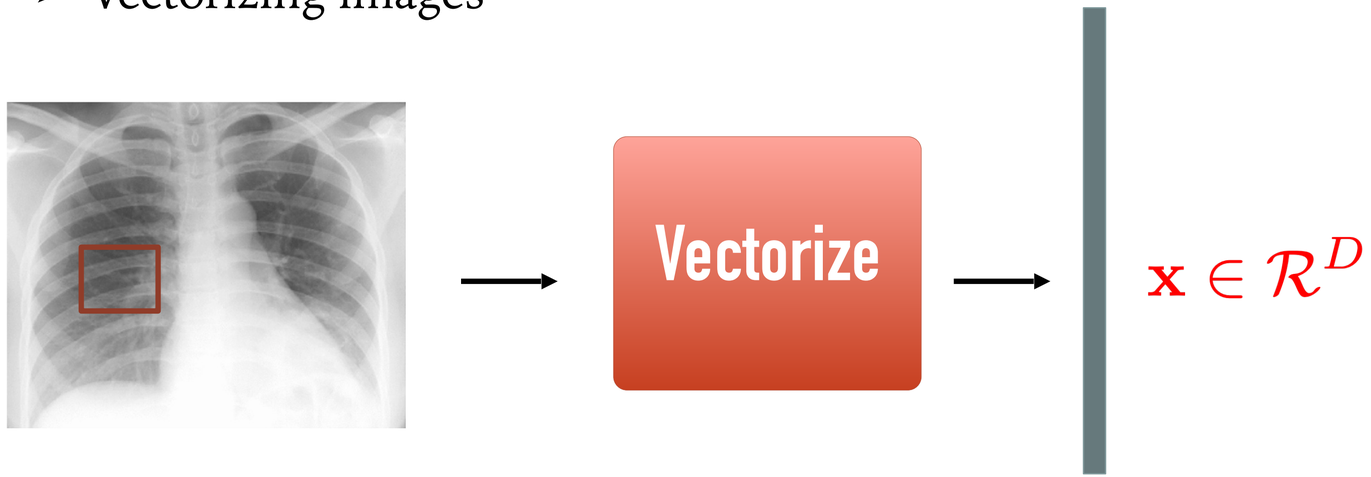
COMPARING DIFFERENT NORMALIZATION

- Batch normalization normalizes each feature independently across the mini-batch. Layer normalization normalizes each of the inputs in the batch independently across all features.
- As batch normalization is dependent on batch size, it's not effective for small batch sizes. Layer normalization is independent of the batch size, so it can be applied to batches with smaller sizes as well.
- Batch normalization requires different processing at training and inference times. As layer normalization is done along the length of input to a specific layer, the same set of operations can be used at both training and inference times.

NEURAL NETWORK ARCHITECTURES

WHAT MAKES DNN SUBOPTIMAL FOR IMAGES

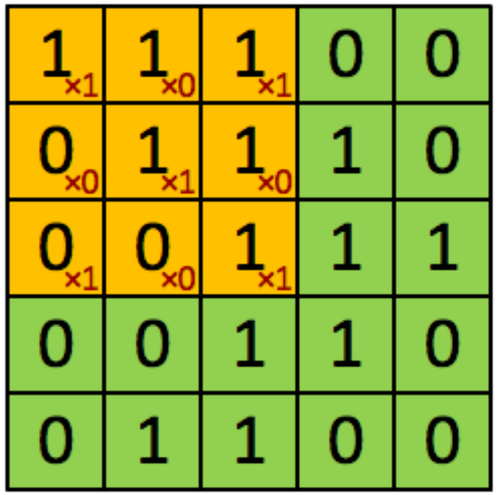
- Vectorizing images



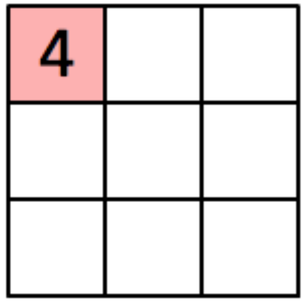
- Ignores the local correlations in the pixels
  - geometric structure is not exploited in images

CONVOLUTIONAL NEURAL NETWORKS

- 2-D convolution $$A^1(i,j) = \sum_{m=0}^{M} \sum_{n=0}^{N} W^1(m,n) X(i+m, j+n)$$



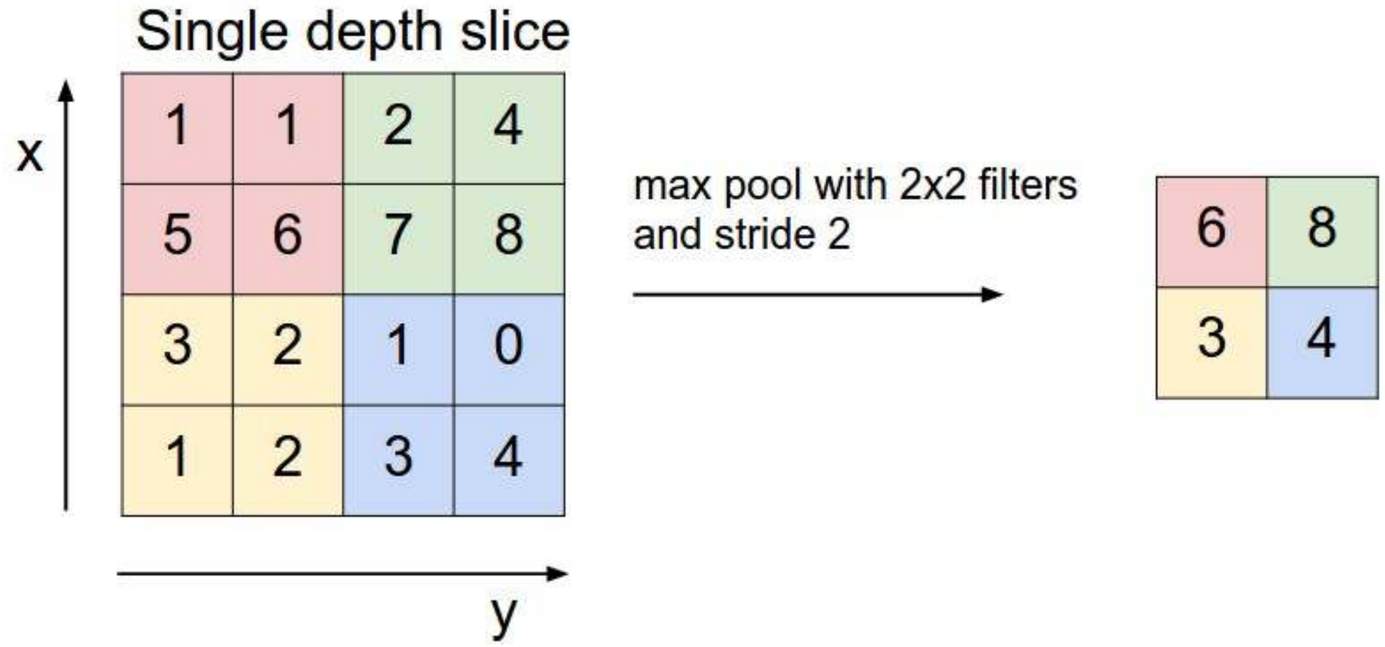
Image



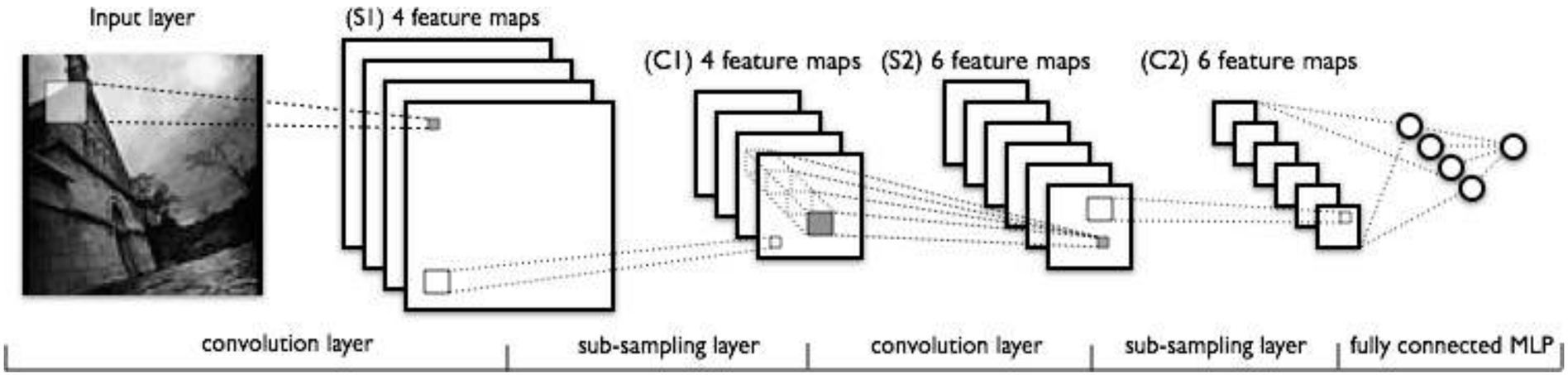
Convolved Feature

CONVOLUTIONAL NEURAL NETWORKS

- Reduce the size of images after convolution using pooling
  - Keep local maximum



CONVOLUTIONAL NEURAL NETWORKS



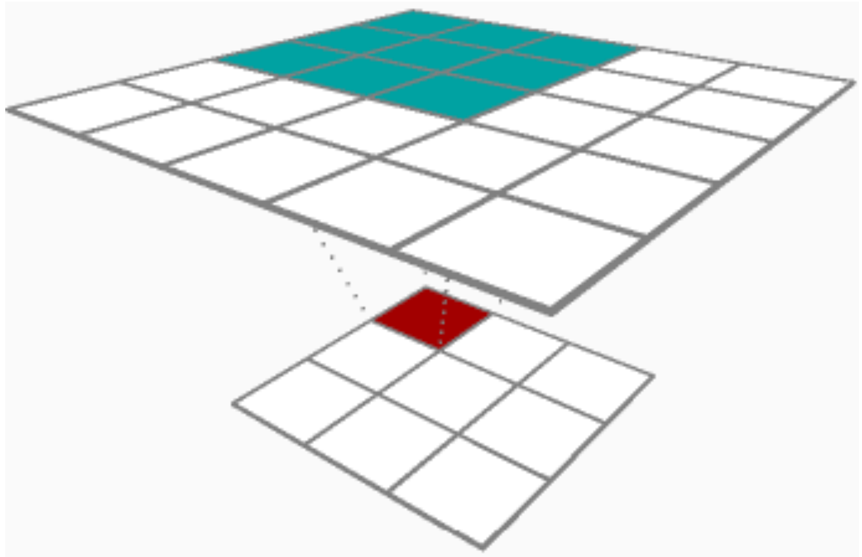
- Multiple levels of filtering and subsampling operations.

- Feature maps are generated at every layer.

BACKPROPAGATION IN CNNS

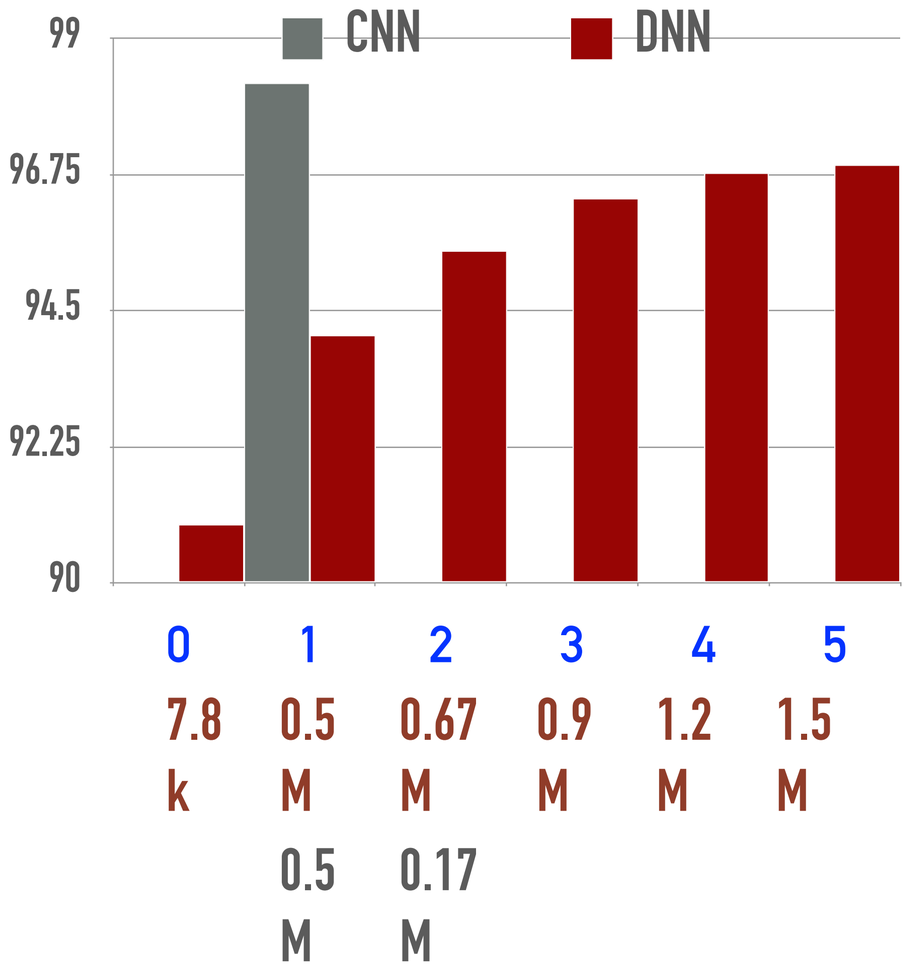
PROPERTIES OF CNN

- Reduce number of parameters
  - due to weight sharing.
  - Depth does not necessarily increase the parameter size.
- Preserving local structure
  - CNN filters operate on local weights
  - Deeper layers
    - capture wider input context.
- Training is more memory intensive
  - Accumulate gradients.



CNNS FOR MNIST

- Providing the right architecture
  - Improves the performance
  - also reduces the number of trainable parameters



UNDERSTANDING DEEP NETWORKS

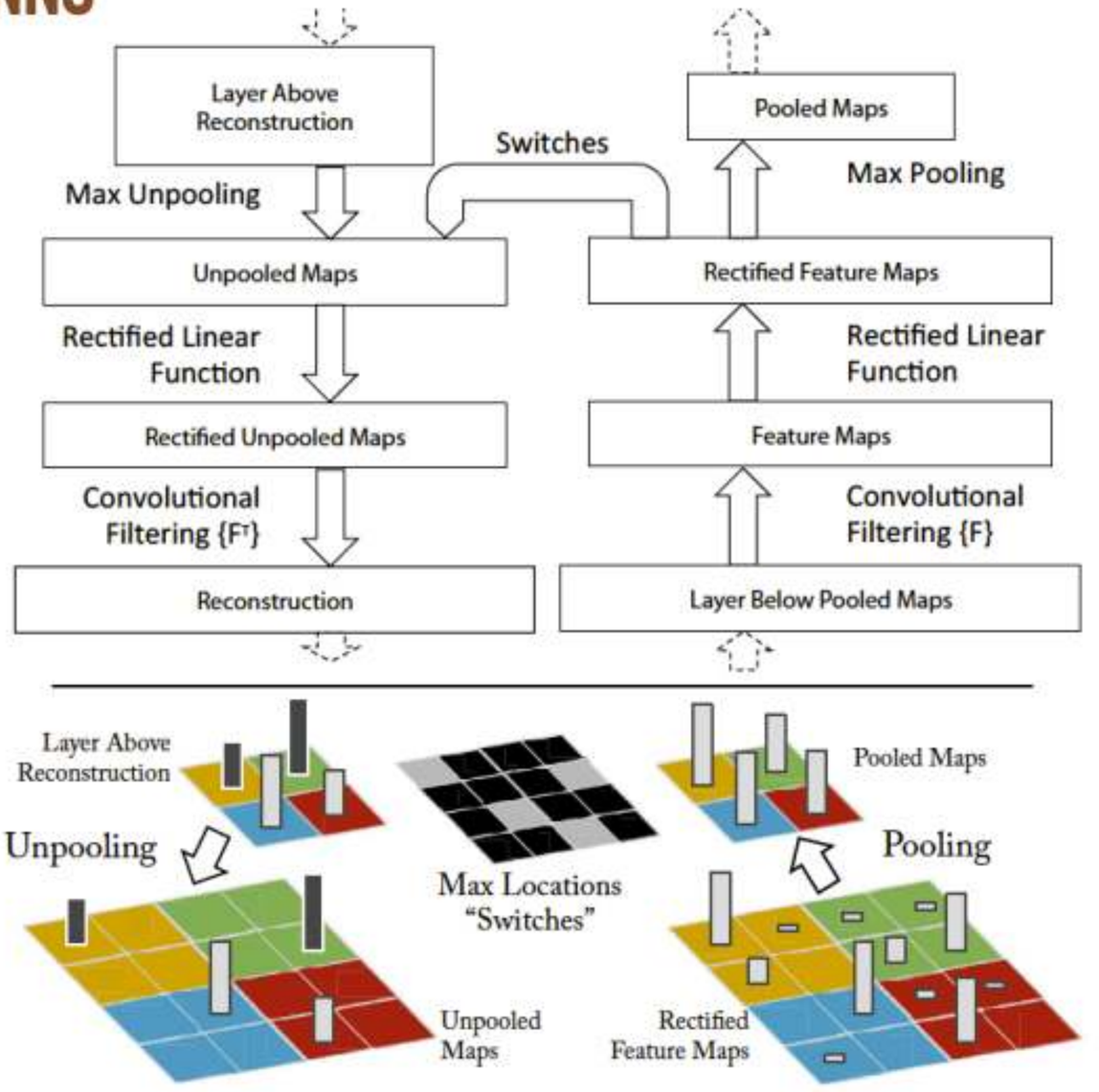
Visualizing and Understanding Convolutional Networks

Matthew D. Zeiler and Rob Fergus

Dept. of Computer Science,
New York University, USA
{zeiler,fergus}@cs.nyu.edu

UNDERSTANDING CNNS USING USING CNNS

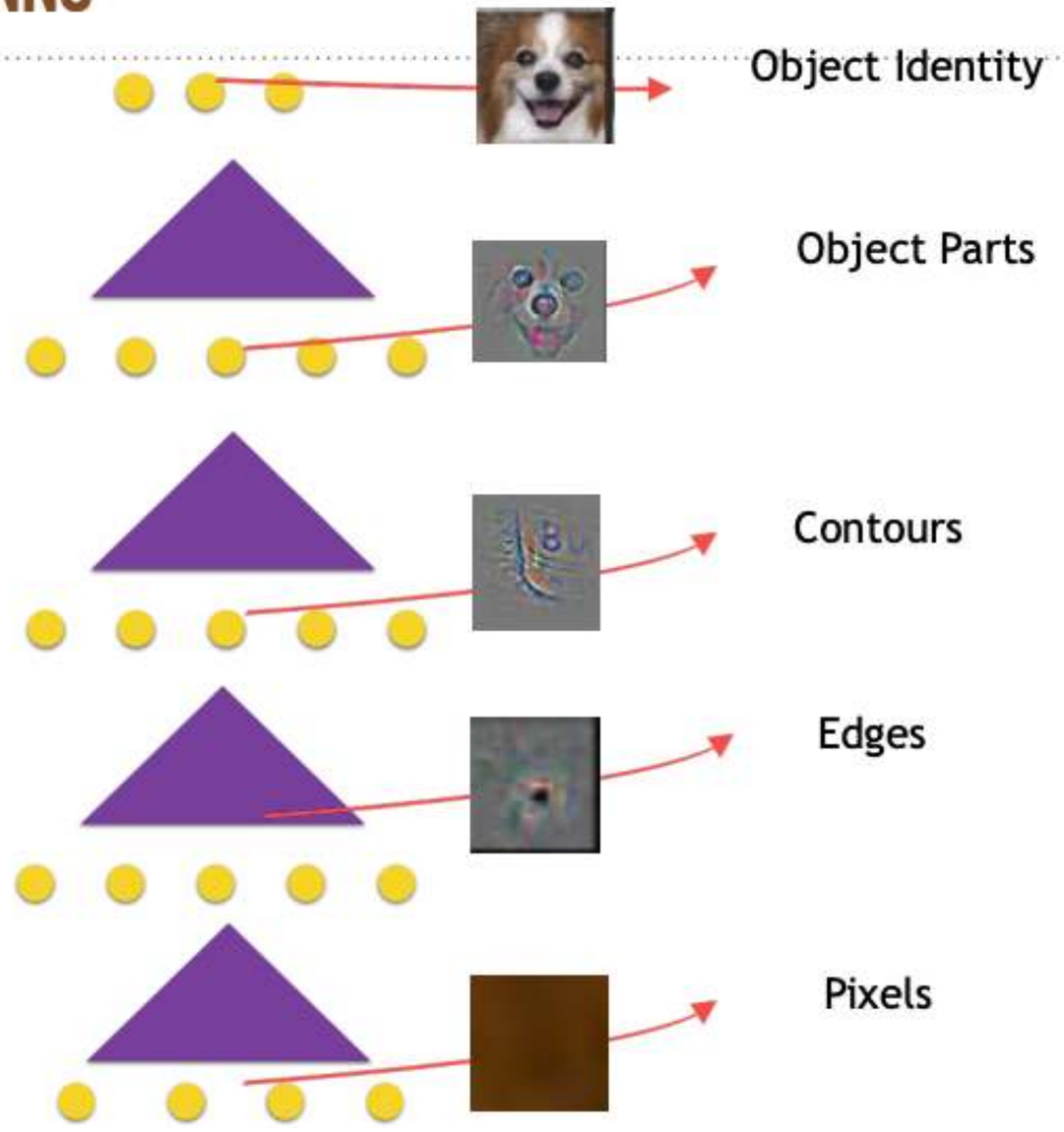
- Analyze a trained model
  - Take a model which is trained to perform object classification
  - Map the layer outputs back to the original space of images
  - Analyze the reconstruction outputs on a held out data



UNDERSTANDING CNNS USING USING CNNS

- Model learns a hierarchy
  - Earlier layers perform simpler tasks like edge detection.
  - Final layers perform complex tasks like object parts

- **Deep networks perform hierarchical data abstraction.**



RECURRENT NEURAL NETWORKS

INTRODUCTION

- The standard DNN/CNN paradigms
  - (x,y) - ordered pair of data vectors/images (x) and target (y)
- Moving to sequence data
  - (x(t),y(t)) where this could be sequence to sequence mapping task.
  - (x(t),y) where this could be a sequence to vector mapping task.
    - Input features / output targets are correlated in time.
    - Unlike standard models where each pair is independent.
    - Need to model dependencies in the sequence over time.

WHY DO NEED RECURRENT MODELS

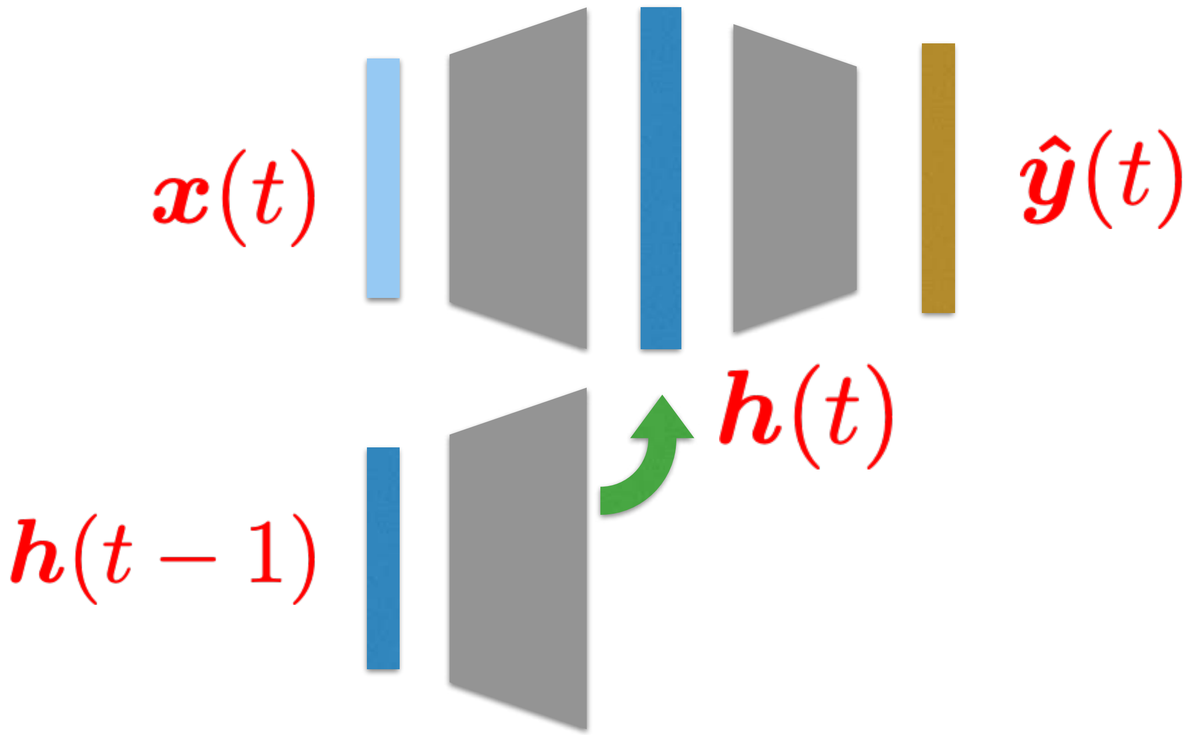
- An interesting subset of this problem is where the input alone is a time series or have different indices $\boldsymbol{x}(t), \boldsymbol{y}$

- Examples
  - Text sequences
  - Speech and audio
  - Video sequences
  - ECG/EEG data
  - Wearable sensor data

FIRST ORDER RECURRENCE - HIDDEN LAYER

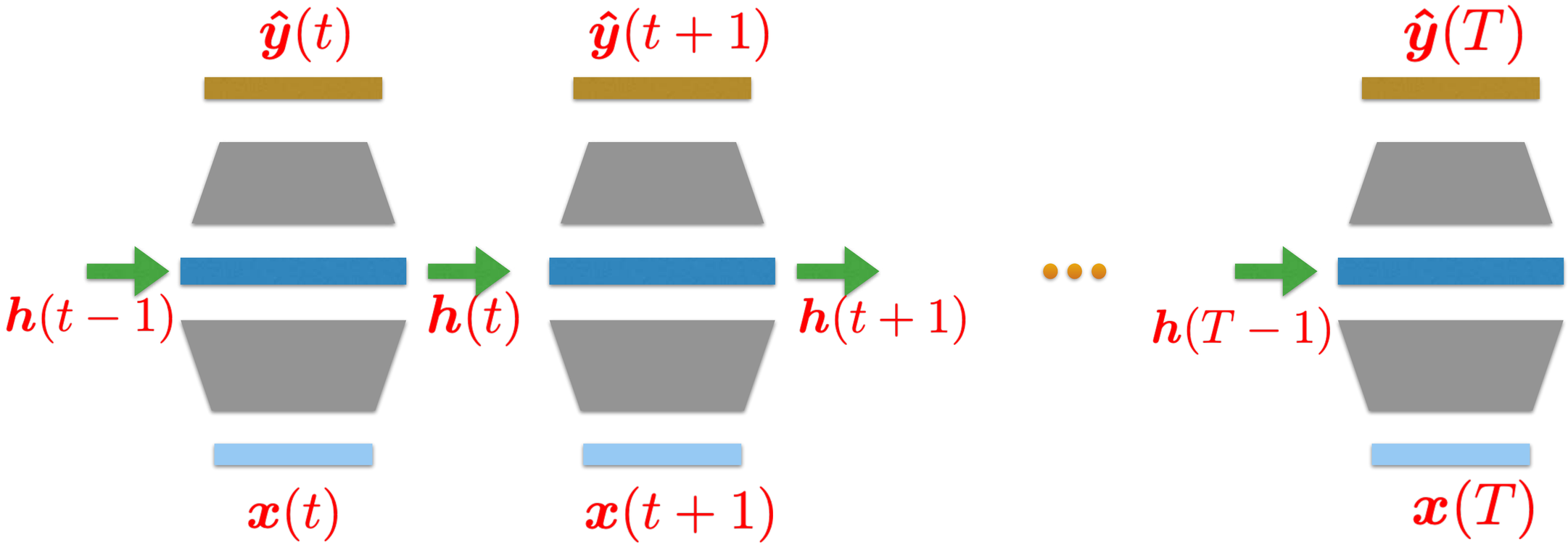
- Making the hidden layer a function of the previous outputs from the hidden layer along with the input

$$\boldsymbol{h}(t) = f\big(\boldsymbol{h}(t-1), \boldsymbol{x}(t)\big)$$



FIRST ORDER RECURRENCE - HIDDEN LAYER

- Making the hidden layer a function of the previous outputs from the hidden layer along with the input.

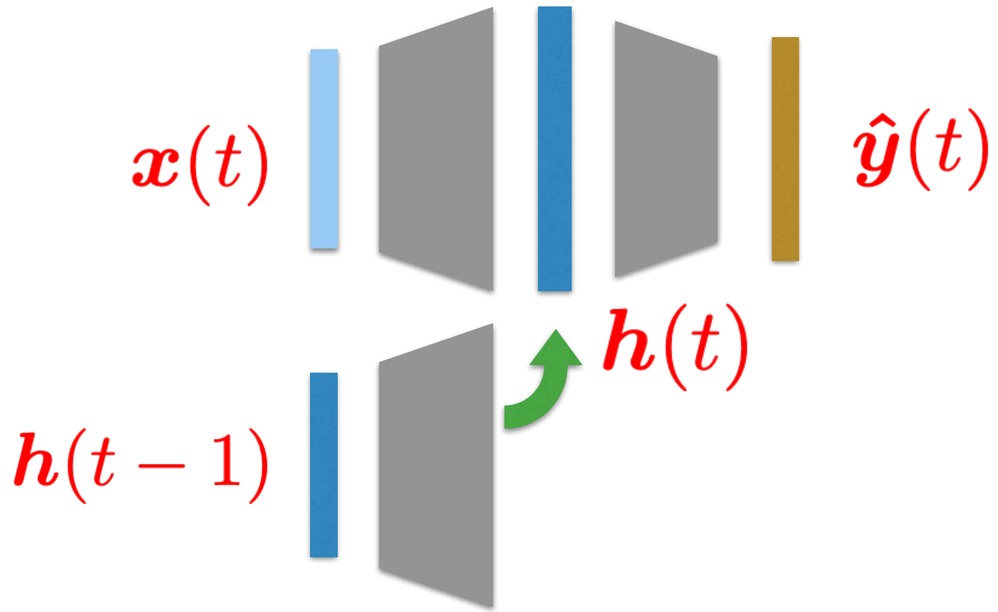


- Makes the hidden layer dependent of previous layer outputs in a recurring fashion.

FIRST ORDER RECURRENCE – HIDDEN LAYER

- Making the hidden layer a function of the previous outputs from the hidden layer along with the input.

$$\boldsymbol{h}(t) = f\big(\boldsymbol{h}(t-1), \boldsymbol{x}(t)\big)$$



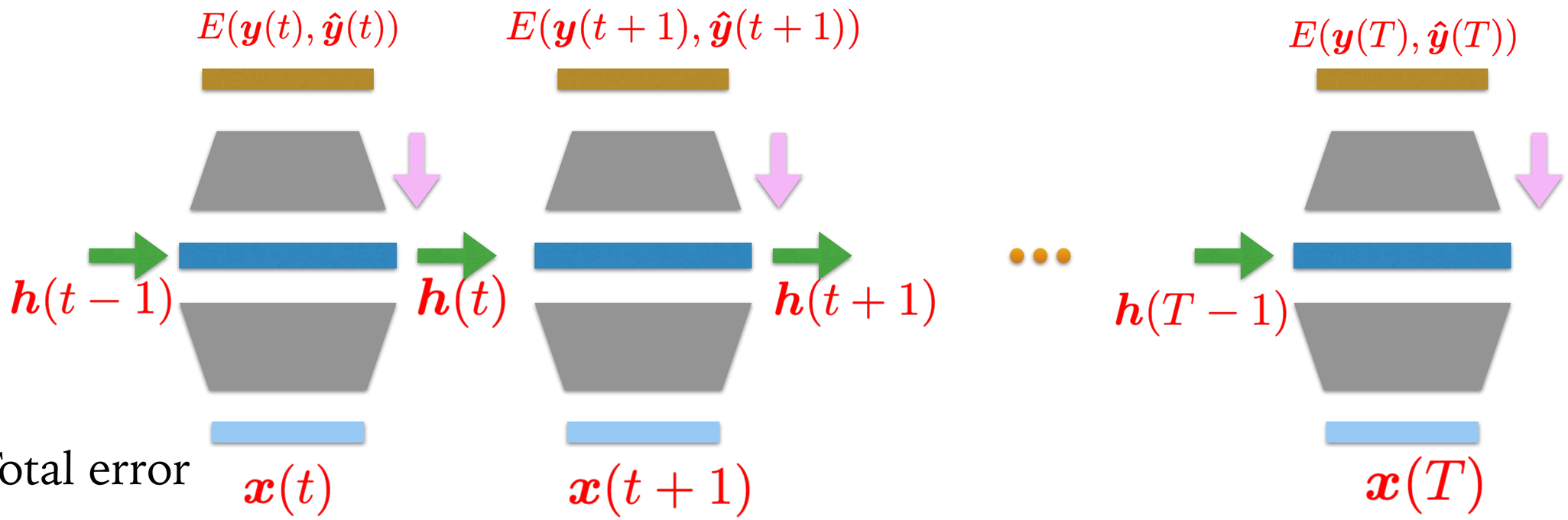
Model Forward Pass - 1 hidden layer

$$\boldsymbol{a}^1(t) = \boldsymbol{W}^1\boldsymbol{x}(t) + \boldsymbol{U}^1\boldsymbol{h}^1(t-1)$$
$$\boldsymbol{h}^1(t) = tanh(\boldsymbol{a}^1(t))$$
$$\boldsymbol{a}^2(t) = \boldsymbol{W}^2\boldsymbol{h}^1(t)$$
$$\hat{\boldsymbol{y}}(t) = S(\boldsymbol{a}^2(t))$$

- Makes the hidden layer dependent of previous layer outputs in a recurring fashion.

ERROR BACKPROPAGATION

- Error functions are computed at every time-instant



- Total error

$$E = \sum_t E(\boldsymbol{y}(t), \hat{\boldsymbol{y}}(t))$$

THANK YOU

Sriram Ganapathy and TA team
LEAP lab, C328, EE, IISc
sriramg@iisc.ac.in